Distributed and Federated Learning

Lab - FL based Anomaly detection with integrated Data Pipeline System

Instructors
Naresh Modina
Masoud Baharlouei
Yasmine Chaouche
ROC-CEDRIC

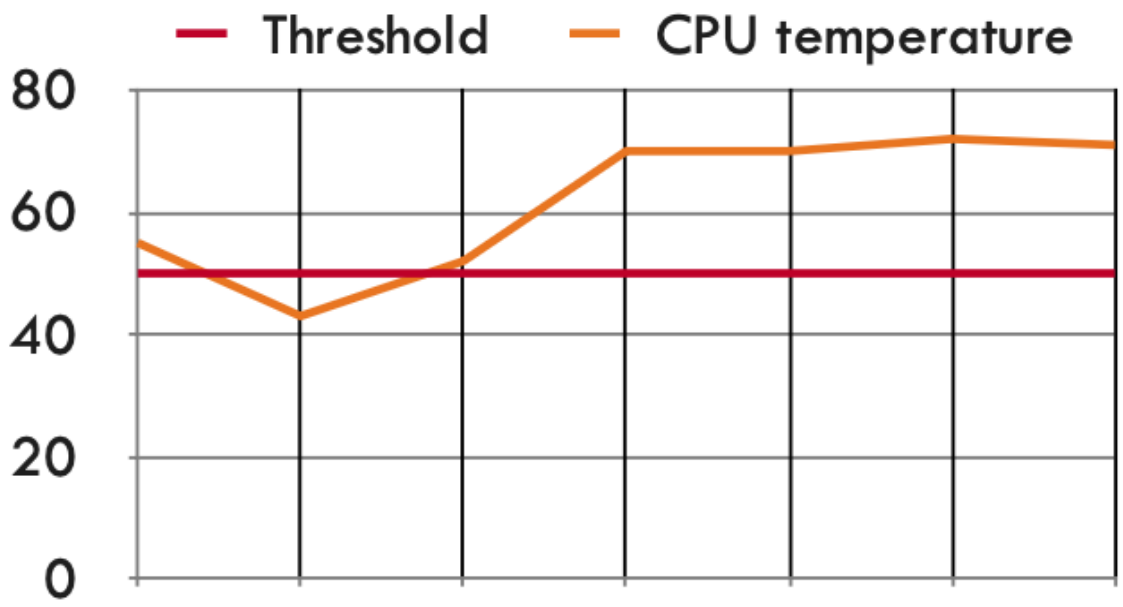
Conservatoire national des arts et mètiers (cnam)
France

Slides (partial) : Patient Ntumba

Outline

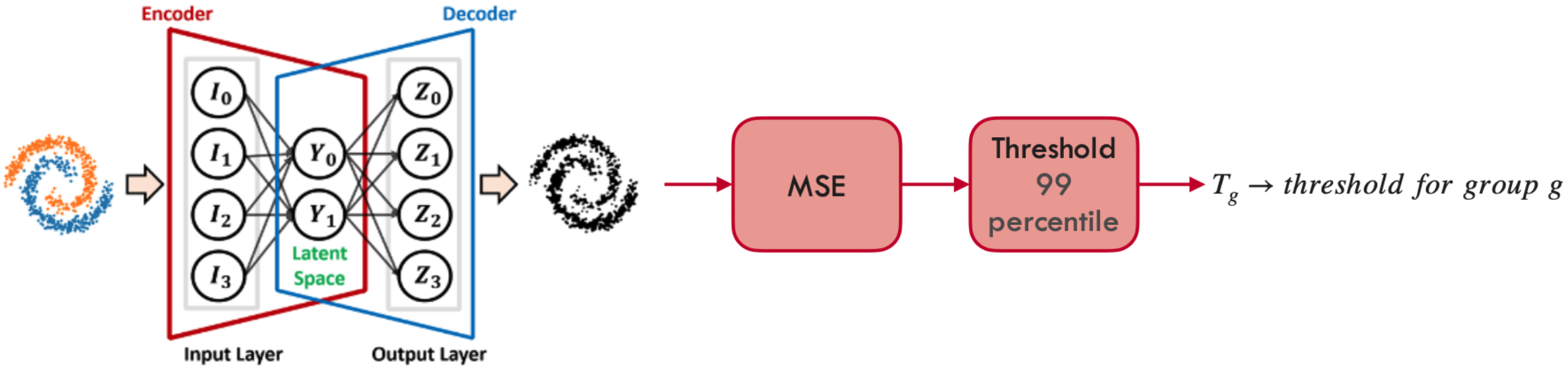
- Anomaly Detection (AD) framework
- Data Pipeline System (DPS)
- Integration of AD and DPS
- Implementation steps
- Tasks to be implemented

Anomaly detection



Any considerable deviation from a normal working condition is characterised as an anomaly

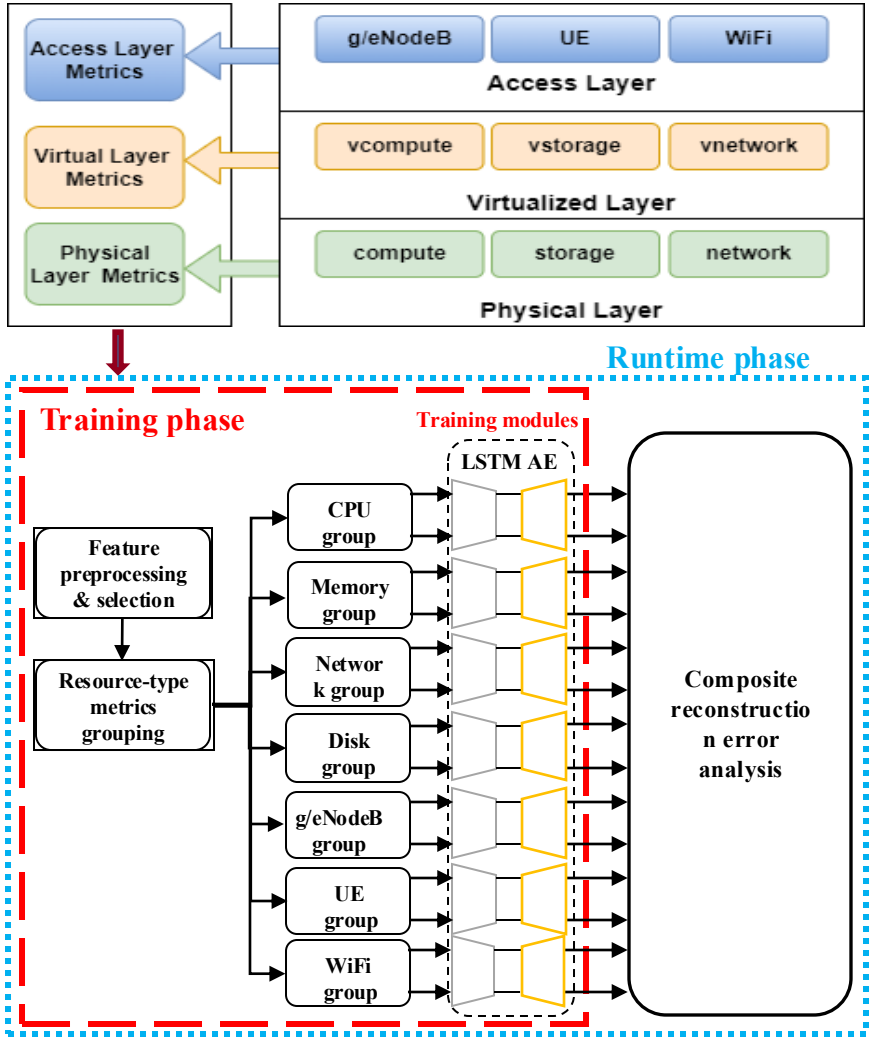
Anomaly detection framework



A. Diaman+, J. M. S. Vílchez and S. Secci, "An AI-Empowered Framework for Cross-Layer Softwarized Infrastructure State Assessment," in IEEE Transactionson Network and Service Management, vol. 19, no. 4, pp. 4434-4448, Dec. 2022, doi: 10.1109/TNSM.2022.3161872.

Anomaly detection framework

- The goal is to find anomalies in a stream of time series (monitoring metrics)
- Staked LSTM autoencoders
  - learn both short and long-time correlations and seasonality in time-series to capture anomalies.
- Metrics are captured from heterogenous sources at different layers
  - Grouped by type of resources.
  - A dedicated autoencoder per group of metrics

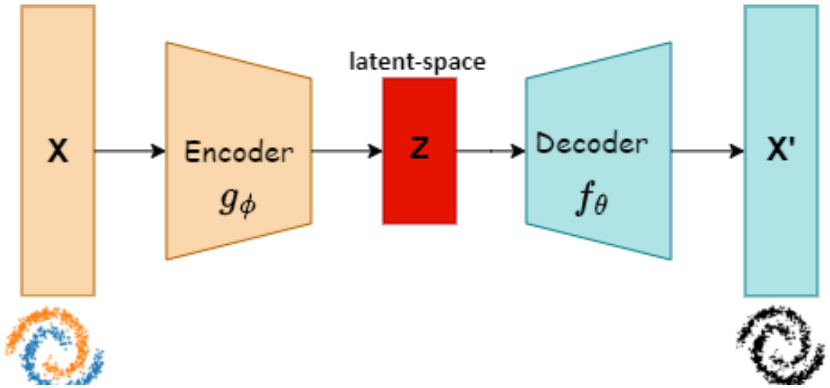


Autoencoders

- An autoencoder (AE) is an unsupervised, feed-forward multi-layer neural network (NN)
- AE is first trained on anomaly free samples.
- Anomalies are spotted by comparing the reconstruction error (RE) for a sample to a given threshold T.

$$RE = \sum_{j=1}^{p} \|Z_j - I_j\|^2 > \mathbf{T}$$

- T is selected on the error distribution ED of the trained dataset. It is usually the nth percentile of the ED



- Encoder: compress the original high-dimension input into the low –dimension latent space **z**

$$z = g_\emptyset(x)$$

- Decoder: reconstruct input from the latent space, likely with larger output layers.

$$x' = f_\theta(g_\emptyset(x))$$

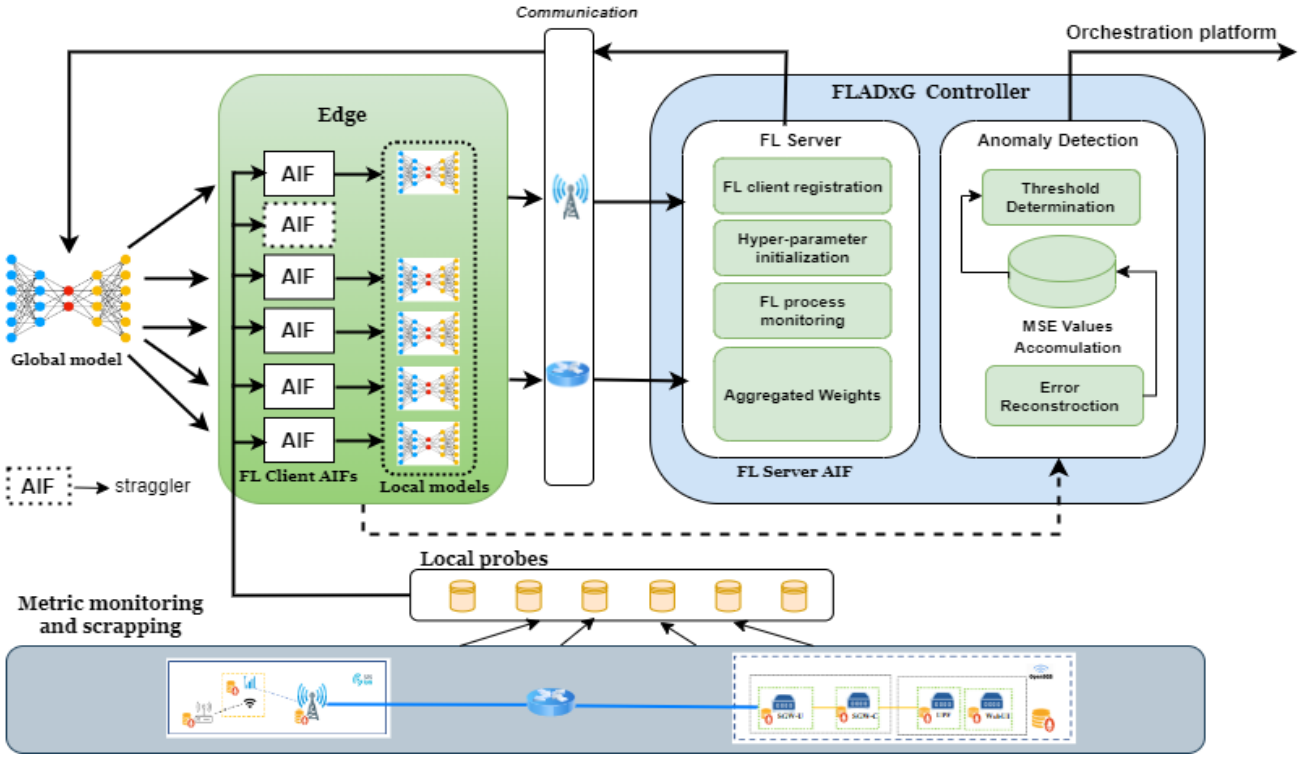
- Both $(\theta, \emptyset)$ are trained together to output a reconstructed data samples almost as the original input.

$$x \approx f_\theta(g_\emptyset(x))$$

Distribution of the training core with FL

- FLADxG: FL deployment for real-time analysis
  - Training LSTM AE in a distributed manner
  - Using edge AIFs to implement the LSTM AEs
  - Iteratively aggregating model parameters
- Minimization of the global loss function (FedAvg)

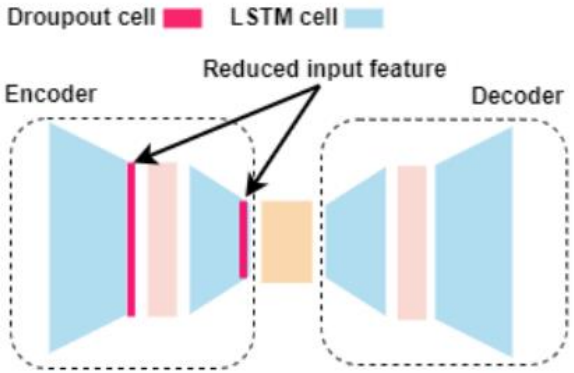
$$f(\mathbf{w}) = \sum_{k=1}^{k} \frac{N_k^{l,g}}{N^{l,g}} f_k(\mathbf{w})$$



S. B. Ruba, N. E.-H. Yellas, and S. Secci, “Anomaly detection for 5g softwarized infrastructures with federated learning,” in 6GNet 2022

Framework Design Consideration

- Factors that can affect the quality of learning:
  - Autoencoder design
  - Hyper-parameters
- Decoders and encoders have 2 cells of LSTM and a dropout layer to prevent over-fitting.
- Hyper-parameters are selected on trail and error to achieve good performance and learning quality

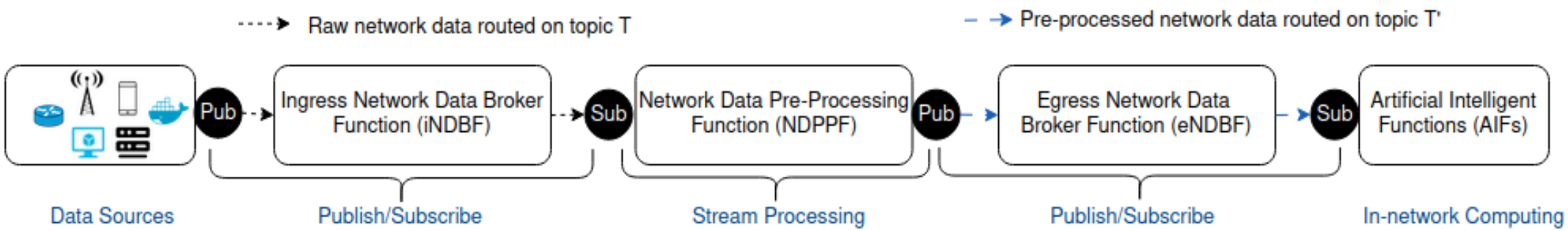


Hyper-parameters	Values
Optimizer	Adam
Loss function	MSE
Dropout rate	0.2%
Learning rate	0.01
Batch size	64
Number of epochs	50
Number of rounds	[2016, 1033, 627]
LSTM Activation function	elu
LSTM recurrent activation function	sigmoid
FL node sampling	100%
Number of participating FL	2, 8, 16, 32

Data Pipeline System Design

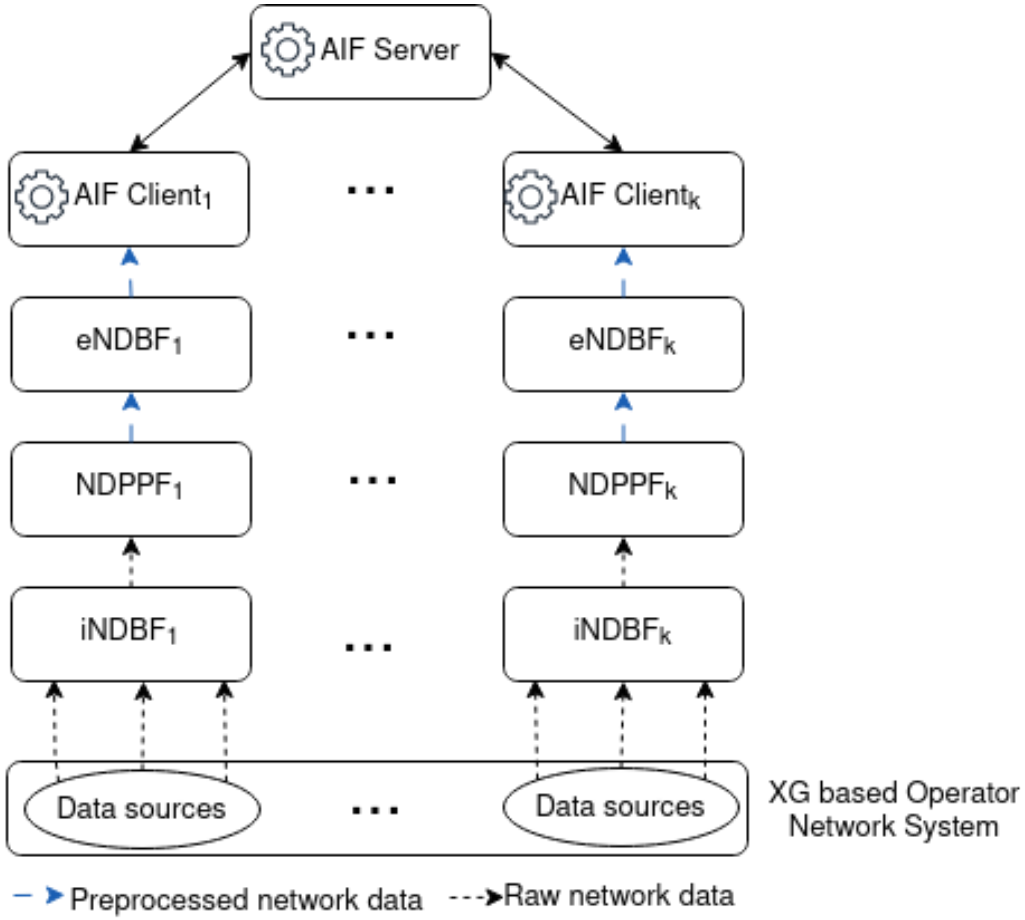
Space, time, synchronisation decoupling

- Awareness of the distributed design of operator network



- Topic-based Publish/Subscribe to route network data from specific source to specific AIF
- Ingress/egress NDBF for queuing network data on the fly until it is fetched
- NDPPF integrates on-the fly pre-processing of network data using stream processing

Possible integration of DPS with AIFs



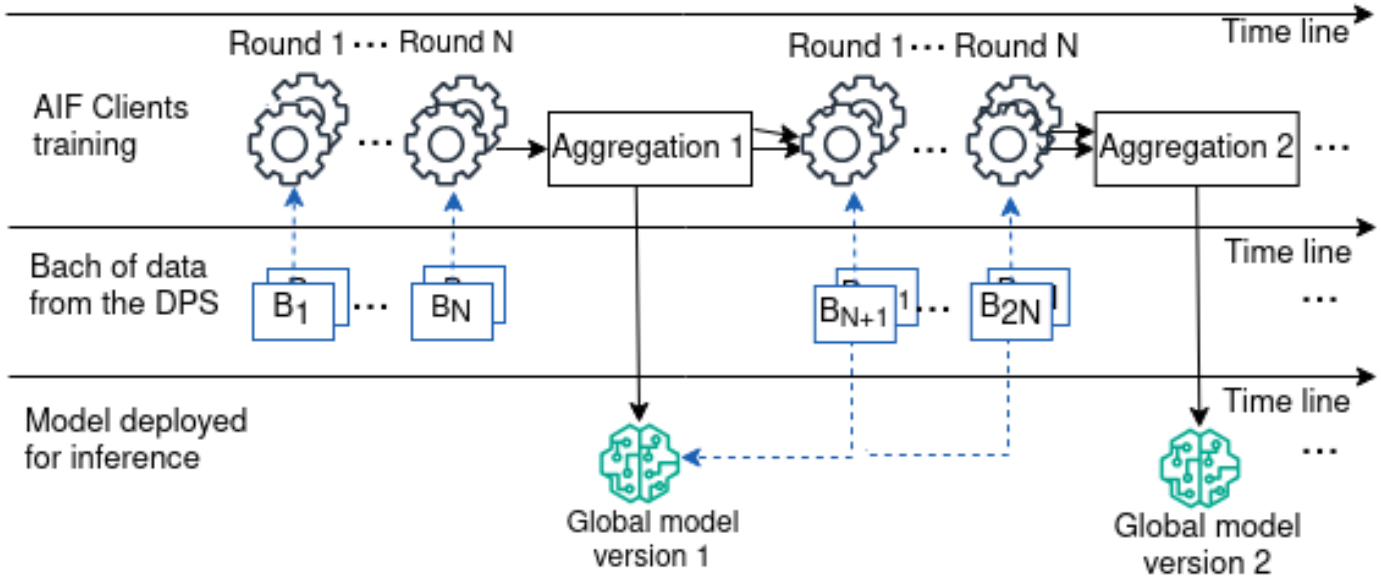
Federated Learning application for anomaly detection

- Assign a DPS function chain to an AIF client

Key challenges

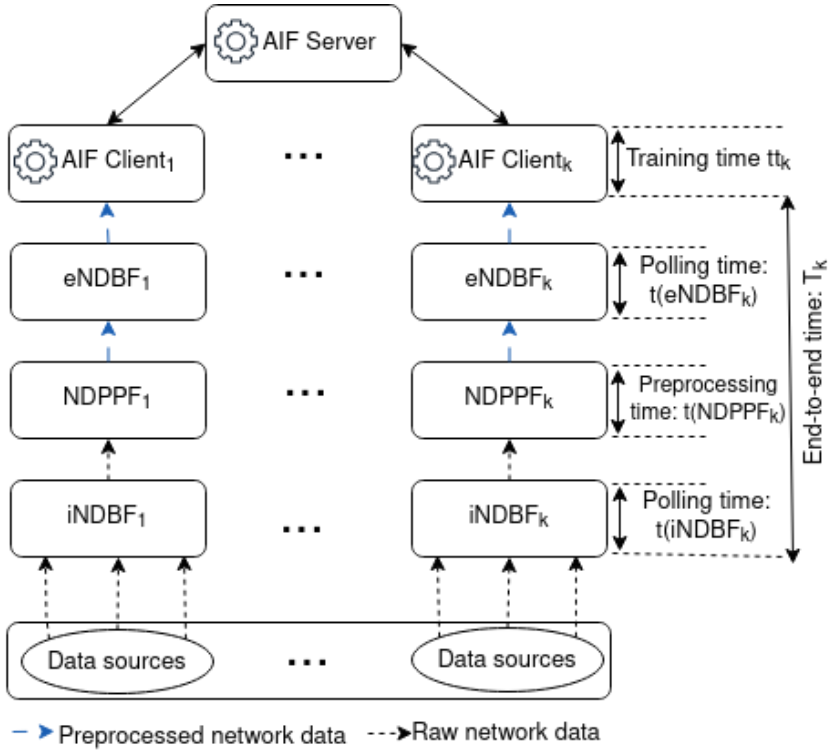
- DPS functions and AIFs choreography
  - Learning/inference on network data in real time
- Deployment of DPS functions and AIFs in operator network
  - Low end-to-end time of network data
- Quality of the trained global model

DPS functions and AIFs choreography

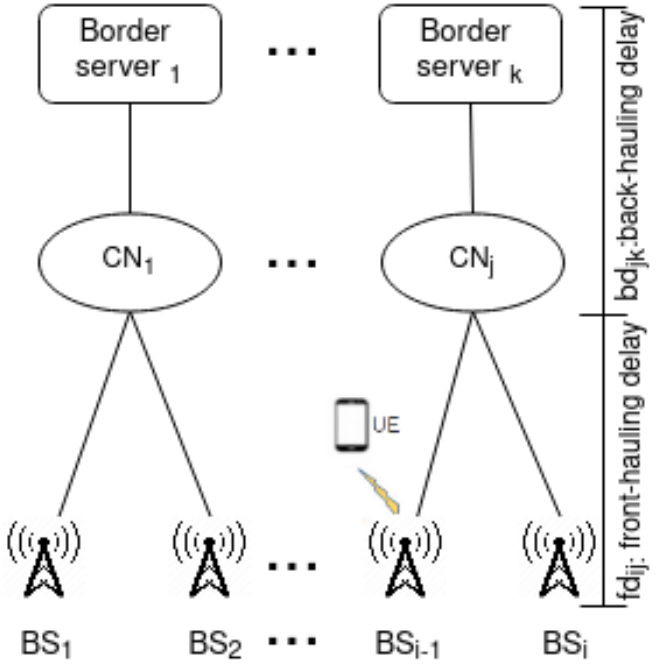


- Each DPS function chain delivers network data batch B_i to the AIF client
- Each AIF client learns from incoming batch B_i as Round i
- The AIF server aggregates learning results after N rounds to produce a global model version
- The global model version is replicated for each DPS function chain for inference

Intregated DPS and AIFs, operator network topology



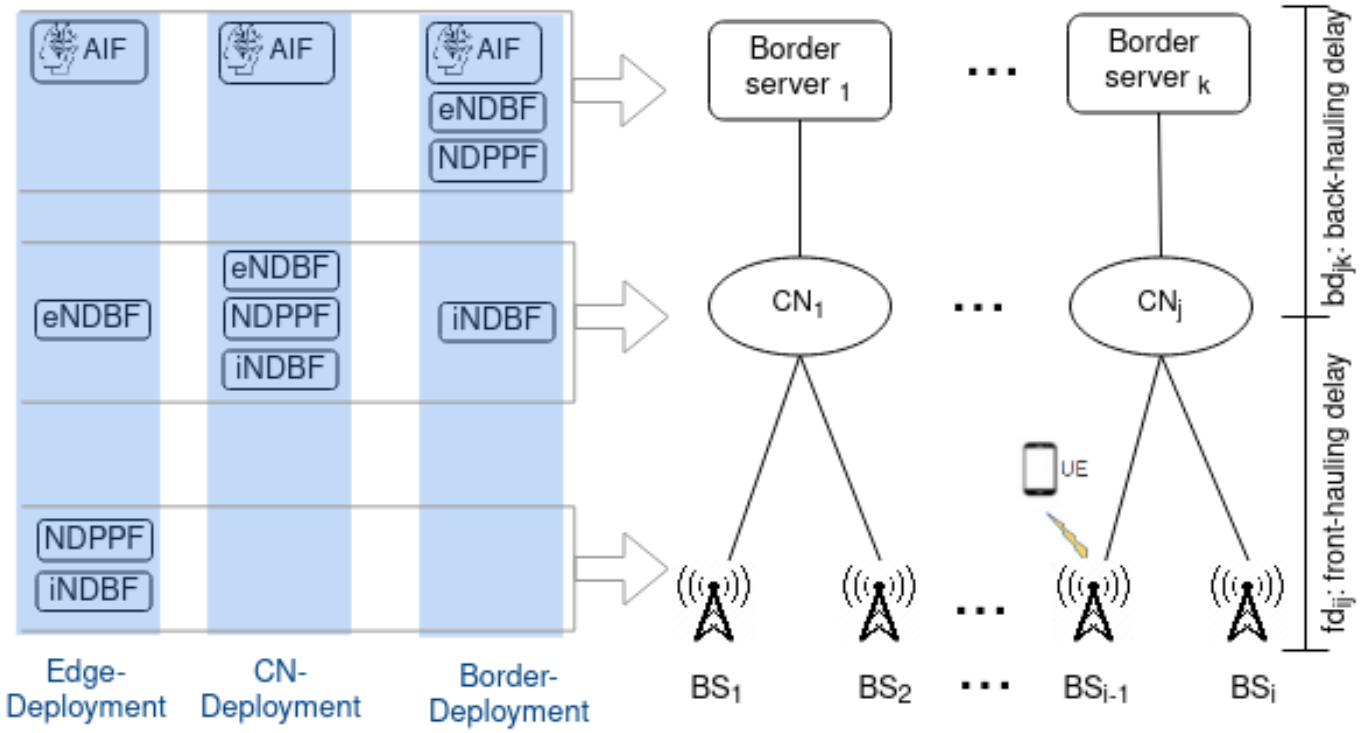
Deploying



- End-to-end time T_k :
  - polling times,pre-processing time ,
  - max(front-hauling delays) and back-hauling delay

- 1 Core network (CN)
  - per set of base stations (BS) and user equipments (UE)
- Border server between the CN and the internet

Deployment setups of DPS in operator network topology



(a) Deployment setups of DPS functions

(b) Operator Network rich view

- Deploy the AIFs at the core network (CN) border
- DPS function placement:
  - Borders
  - Core network (CN)
  - Closer to the data sources (Edge)
- Three deployment setups
  - Border-deployment
  - CN-deployment
  - Edge-deployment

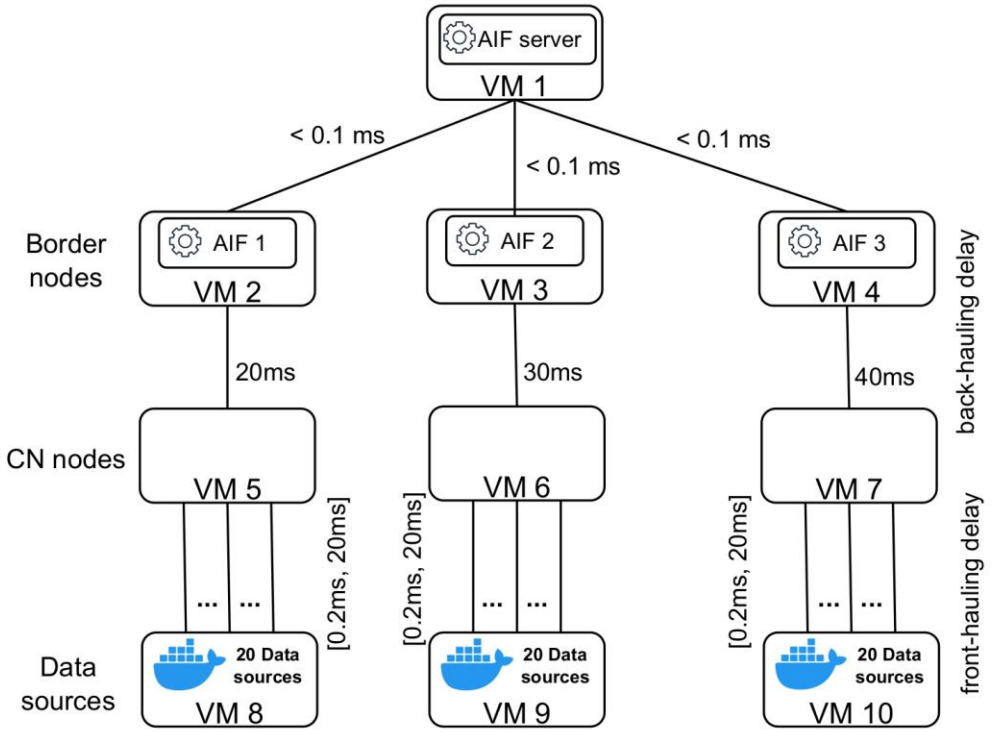
Setting of the Data sources and DPS

Monitored network data

- 60 container docker as data sources
  - 26 features of CPU/Memory related metrics
  - Sampling rate 100ms
- Evenly distribute 20 DS per DPS function chain

3 DPS function chains

- Use Apache Kafka for i/eNDBF
- NDPPF processes network data in batch of 200 samples



- Anomalous joint CPU/memory usage

Tasks to be completed

Task 1: Run the setup with 2 clients to detect anomalies

Task 2: Load balancing with 3 clients and 1 data brokers and 6 data sources

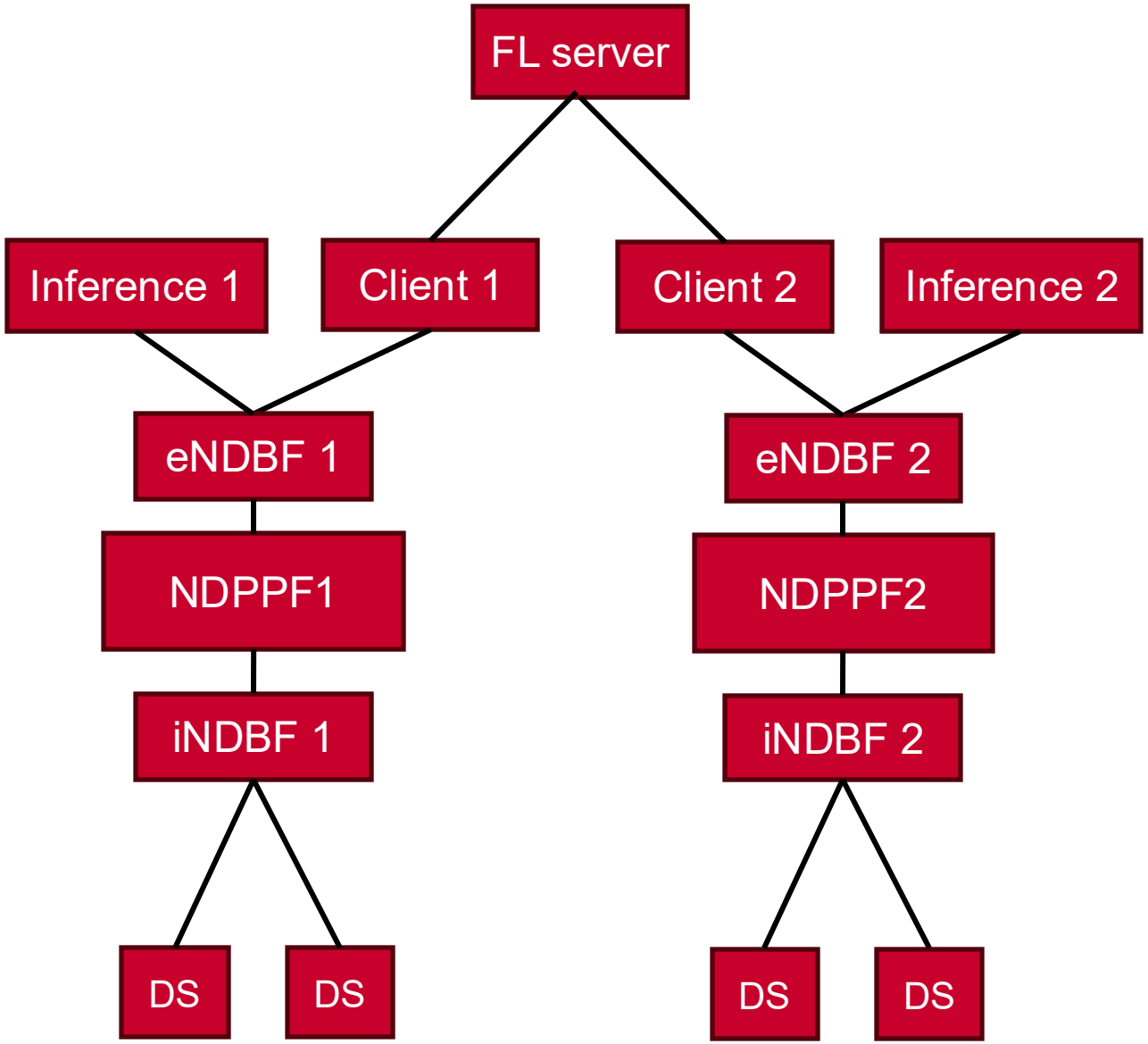
Task 3: Implement and compare three averaging techniques (FedAvg, FedProx [3], SCAFFOLD [2])

Task 1

- Follow the implementation details as per the readme file in the following gitlab repo

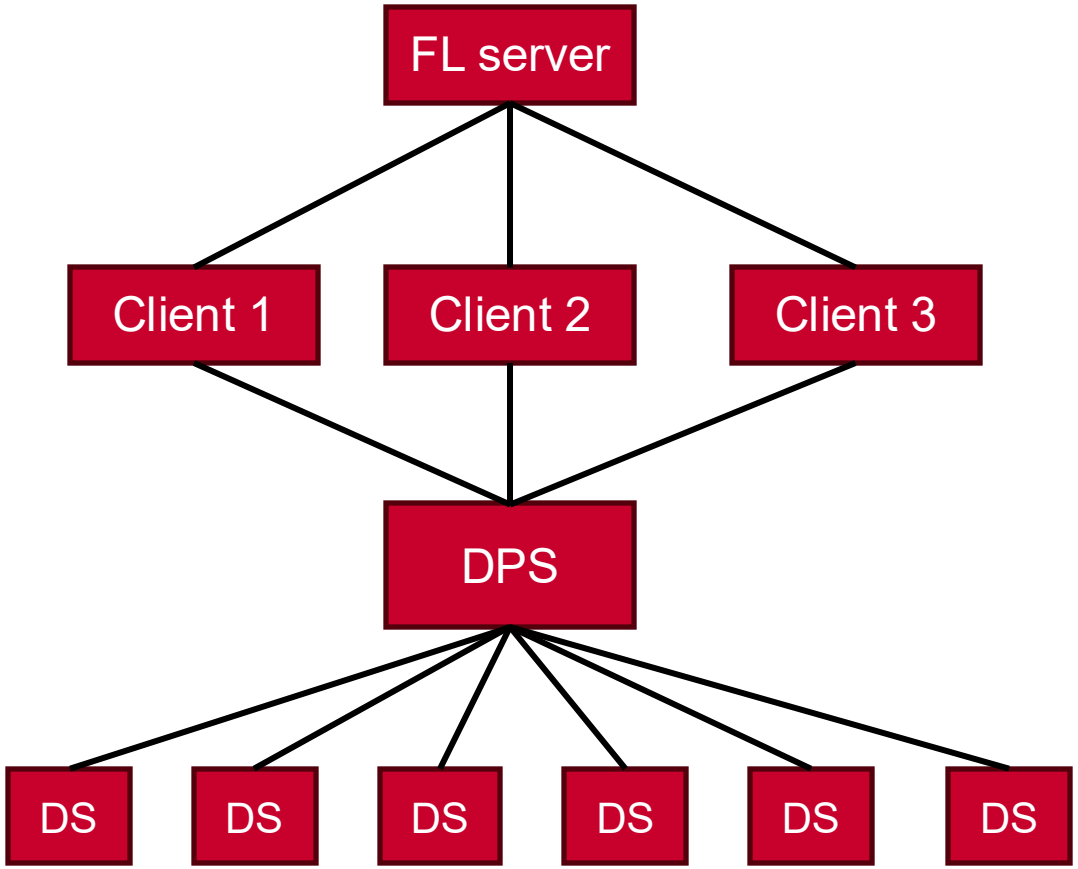
  https://gitlab.roc.cnam.fr/goyban/usees3_fladdps

Task 1: Topology



Task 2: topology

- Implement the topology shown in the figure, in which all 6 data sources are connected to a single DPS, which in turn is connected to all 3 clients.



Task 2: Load balancing

- After setting up the topology, you are asked to implement two scenarios in which the DPS performs data load balancing:

  Scenario 1:

  The DPS distributes data uniformly to all clients, so that each client gets the same data distribution.

- **Scenario 2:**

  The DPS sends 20% of the total collected data to Client 1, 30% to Client 2, and 50% to Client 3.

Task 3

- Implement both the SCAFFOLD **[2]** and FedProx **[3]** algorithms and plot the training loss over communication rounds.
- Compare the results with FedAvg in terms of performances (loss, convergence speed...) and interpret the findings.

Instructions:

- Complete Tasks 1 and 2 by the end of the day.
- Write a report explaining all implementation steps, the results, and their interpretation (provide screenshots of the results).
  - Write your code with proper comments.
- Submit both your **report** and **code** on moodle before **13 june 2025, 18:00**

REFERENCES

[1] Patient Ntumba, Nour-El-Houda Yellas, Salah Bin Ruba, Fehmi Ben Abdesslem, Stefano Secci. Data Pipeline System Designs for In-network Learning. *20th International Conference on Network and Service Management Prague, Czech Republic // 28 - 31 October 2024*, Oct 2024, Prague, Czech Republic. ⟨hal-04681121⟩

[2] Karimireddy, S.P., Kale, S., Mohri, M., Reddi, S., Stich, S. and Suresh, A.T., 2020, November. Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning* (pp. 5132-5143). PMLR.

[3] Li, T., Sahu, A.K., Zaheer, M., Sanjabi, M., Talwalkar, A. and Smith, V., 2020. Federated optimization in heterogeneous networks. Proceedings of Machine learning and systems, 2, pp.429-450.